

Protein Circuit Topology Plugin - Complete API Documentation

Total Callable Entry Points: 182 **Python Files With Callables:** 41

Table of Contents

1. [Calculating Functions](#) (7 functions)
2. [Plotting Functions](#) (9 functions)
3. [Importing Functions](#) (2 functions)
4. [Exporting Functions](#) (3 functions)
5. [Analysis Functions](#) (14 functions)
6. [Utility Functions](#) (66 functions)
7. [GUI Functions](#) (67 functions)
8. [Initialization Functions](#) (14 functions)

Calculating Functions

`energy_cmap(index, numbering, res_names, protid, potential_sign='-')`

Module: functions/calculating/energy_cmap.py

Applies an energy filter to an existing residue contact map based on a potential matrix.

`_atoms_for_search(atoms, include_hydrogens)`

Module: functions/calculating/get_cmap.py

Heavy atoms only, unless the caller asked for ProteinCT-identical all-atom counting.

`get_cmap(chain, level='chain', cutoff_distance=4.5, cutoff_numcontacts=5, exclude_neighbour=3, include_hydrogens=False)`

Module: functions/calculating/get_cmap.py

Creates a residue-residue contact map (as a list of contacts) for a single chain or a whole model.

Two residues are in contact when at least `cutoff_numcontacts` atom-atom pairs lie within `cutoff_distance`. Hydrogen atoms are ignored by default, so models with and without hydrogens give the same contacts; `include_hydrogens=True` counts every atom present, which is what the reference ProteinCT implementation does. At level 'model' the whole model is searched, so inter-chain contact pairs are included, and the neighbour exclusion is applied on the concatenated residue index (it therefore also spans a chain boundary).

`get_matrix(index, protid)`

Module: functions/calculating/get_matrix.py

Creates a topological relationship matrix for a residue contact map.

Single chain (two-column index): the relation between every two contact pairs is Series (S), Parallel (P, P-1), Cross (X) or one of the concerted types (CP, CP-1, CS). Whole model (four-column index, chain IDs attached): a relation is S, P or X only when both contact pairs lie within one and the same chain.

Every other relation, i.e. one in which at least one contact pair joins two chains, uses the multi-chain vocabulary of Heidari et al. (2022) as adapted in ProteinCT: Independent (I) when the two contact pairs share no chain, Tandem (T) when they share a chain but cannot be joined into a closed path along chains and contacts, and Loop (L) when they can, which for two contact pairs means both join the same two chains. The digit in the statistics labels (I2, I3, I4, T2, T3) is the number of chains the two contact pairs span, the superscript of Heidari et al.'s notation.

`get_stats(mat)`

Module: functions/calculating/get_stats.py

Calculates the fraction of Parallel and Cross (non-Series) relations on each diagonal of the relation matrix, i.e. as a function of the distance between two contact pairs in the contact list. This is the quantity ProteinCT calls the "entangled" fraction.

`length_filter(index, distance, mode='<')`

Module: functions/calculating/length_filter.py

Filters contact indices based on sequence separation distance.

`local_ct(index, mat, numbering)`

Module: functions/calculating/local_ct.py

Calculates local circuit topology statistics for each residue.

Plotting Functions

`show(fig)`

Module: functions/plots/_display.py

Display a figure without blocking PyMOL, regardless of what ran before it.

`discrete_cmap(colors)`

Module: functions/plots/_palette.py

Return (cmap, norm) mapping integer code *i* to colors[*i*], for any subset of codes.

`circuit_plot(index, protid, numbering)`

Module: functions/plots/circuit_plot.py

Plots the circuit topology of a protein as a series of arcs.

`local_topology_plot(index, mat, numbering, siteid, relation)`

Module: functions/plots/local_topology_plot.py

`matrix_plot(mat, protid)`

Module: functions/plots/matrix_plot.py

Plots the topological relationship matrix for a single chain.

`matrix_plot_model(mat, protid)`

Module: functions/plots/matrix_plot_model.py

Plots the topological relationship matrix for a whole model (multiple chains).

wedge_colors(count)

Module: functions/plots/stats_plot.py

Colours for the pie, in the order Parallel, Series, Cross, then the multi-chain classes.

stats_plot(px_fraction, psx, protid)

Module: functions/plots/stats_plot.py

Plots the fraction of Parallel and Cross (non-Series) relations versus distance from the diagonal of the relation matrix, and a pie chart of the relation types.

ProteinCT calls the left-hand quantity the "entangled" fraction; it is a statement about relation types, not about geometric entanglement.

autopct_func(pct)

Module: functions/plots/stats_plot.py

Importing Functions

retrieve_chain(input_file, chainid=0)

Module: functions/importing/retrieve_chain.py

Retrieves a specific chain from a PDB or MMCIF file.

warn_about_numbering(chain, protid)

Module: functions/importing/retrieve_chain.py

Report the two residue-numbering situations that contact detection handles imprecisely.

Exporting Functions

export_cmap3(index, protid, numbering, output_dir)

Module: functions/exporting/export_cmap3.py

Exports a residue contact map (as a binary matrix) to a CSV file.

export_mat(index, mat, protid, output_dir)

Module: functions/exporting/export_mat.py

Exports the topological relationship matrix to a CSV file.

export_psx(psxlist, output_dir)

Module: functions/exporting/export_psx.py

Exports the counts of Parallel (P), Series (S) and Cross (X) relations (and, for multi-chain models, I/T/L) to a CSV file.

A batch run accumulates every input's row into one psxlist and calls this once, so a single output file is correct - there is nothing to disambiguate with a per-input name.

Analysis Functions

run_local_ct(self)

Module: analysis/local_ct_analysis.py

Runs the local circuit topology analysis based on user-selected parameters. Handles data retrieval, calculation, plotting, and exporting.

_states_of(traj_obj, n_states)

Module: analysis/multiple_file_analysis.py

One work item per trajectory state, read straight from the loaded object.

run_multi_analysis(self)

Module: analysis/multiple_file_analysis.py

Runs the multi-file circuit topology analysis. Iterates through files in the selected directory, performs analysis, and generates plots/exports.

run_standard_analysis(self)

Module: analysis/single_file_analysis.py

Runs the standard single-file circuit topology analysis. Handles data retrieval, calculation, plotting, and exporting.

run_single_frame_analysis(self)

Module: analysis/single_frame_analysis.py

Runs circuit topology analysis for a single frame of a trajectory or a single PDB file from a directory.

toggle_frame_controls(self, enabled)

Module: analysis/single_frame_analysis.py

Toggles the enabled state of the frame selector and run button.

_safe_delete(*names)

Module: analysis/visualization.py

Delete PyMOL objects/selections, ignoring names that do not exist.

_analyse_chains(target_obj, relation_type, vals, state)

Module: analysis/visualization.py

Run the circuit topology analysis for every chain of a single-state object.

Kept separate from the colouring so that every chain - and every frame of a trajectory - can be measured first and then coloured against one shared scale.

_shared_bounds(analyses)

Module: analysis/visualization.py

One set of bucket bounds covering every chain and frame, so colours are comparable.

_apply_colours(analyses, relation_type, bounds)

Module: analysis/visualization.py

Paint each analysed chain against the shared scale.

`_color_chains_by_topology(target_obj, relation_type, vals, state, *, scale_bar=True)`

Module: analysis/visualization.py

Analyse and colour every chain of a single-state object against one shared scale.

`_color_every_state(state_objs, relation_type, vals, split_prefix)`

Module: analysis/visualization.py

Colour each per-state object, quietly and without repainting the scene N times.

`_visualize_trajectory(self, relation_type, selected_obj, vals, n_states)`

Module: analysis/visualization.py

Colors every state of a trajectory object by its own circuit topology.

`visualize_molecule(self, relation_type)`

Module: analysis/visualization.py

Visualizes the circuit topology on the selected molecule in PyMOL by colouring residues by their participation in the selected relation type.

Utility Functions

`clear_selected_single_file(self)`

Module: utils/clear_file.py

Clears the currently selected single file and updates the UI.

`clear_selected_local_file(self)`

Module: utils/clear_file.py

Clears the currently selected local file and updates the UI.

`_ensure_object_loaded(obj_name)`

Module: utils/directory.py

`_load_structure_file(self, label, attr_file, attr_obj)`

Module: utils/directory.py

Shared helper: open a file dialog, load into PyMOL, check for non-polymer atoms.

`_choose_output_dir(self, attr_name, label)`

Module: utils/directory.py

Shared helper: open a directory dialog and store the result.

`choose_file(self)`

Module: utils/directory.py

`choose_local_file(self)`

Module: utils/directory.py

choose_local_output_dir(self)

Module: `utils/directory.py`

choose_output_dir(self)

Module: `utils/directory.py`

choose_output_dir_multi(self)

Module: `utils/directory.py`

choose_input_dir_multi(self)

Module: `utils/directory.py`

Opens directory dialog to select the input directory containing PDBs for multi-file analysis. Updates the frame selector spinbox based on the number of PDB files found.

set_label_text_elided(file_path, label)

Module: `utils/directory.py`

Sets the text of a QLabel to an elided version of the file path if it's too long.

get_folding_score(mat, index, numbering)

Module: `utils/folding_score.py`

Calculate the folding score based on the given relations, using topology data.

The score is the sum of the mean per-residue participation in Parallel, Series and Cross relations.

get_vis_vals(self)

Module: `utils/get_values.py`

Retrieves visualization parameters from the GUI.

get_values(self)

Module: `utils/get_values.py`

Retrieves parameters for single-file analysis from the GUI.

get_local_values(self)

Module: `utils/get_values.py`

Retrieves parameters for local analysis from the GUI.

get_multiple_values(self)

Module: `utils/get_values.py`

Retrieves parameters for multi-file analysis from the GUI.

update_chain_combo_box(self)

Module: `utils/helpers.py`

Updates the chain combo box with the chains available in the currently selected object.

make_info_button(tooltip)

Module: `utils/helpers.py`

Creates a small info button with a tooltip.

`temp_pdb_export(selection, state=None, label=None)`

Module: `utils/helpers.py`

Save a PyMOL selection to a temporary PDB file, yield the path, then clean up.

`notify(message)`

Module: `utils/helpers.py`

Put a message on PyMOL's command line.

`make_note_row(text, tooltip)`

Module: `utils/helpers.py`

A quiet grey caption with its info button pinned to the right.

`make_param_row(label_text, tooltip, spinbox)`

Module: `utils/helpers.py`

Create a standard parameter row layout with label, info button, and spinbox.

`show_folding_score_dialog(parent, chain, folding_score)`

Module: `utils/helpers.py`

Show the CT folding score for a chain in a small, auto-sized pop-up dialog.

`resolve_output_path(self, output_dir)`

Module: `utils/helpers.py`

Validates and creates the output directory. Returns the Path on success, None on failure.

`report_empty_local_result(idx, mat, residue_id, residue_number, contact)`

Module: `utils/local_ct_report.py`

Print a notice when the plot will highlight nothing; return "" when it will.

`residue_id` is a POSITION into `numbering`; `residue_number` is what the user typed.

`non_polymer_counts(obj_name)`

Module: `utils/non_polymer.py`

Residue-name histogram of the non-polymer atoms in one object. Empty when there are none.

`report_excluded_atoms(obj_name)`

Module: `utils/non_polymer.py`

Log what analysis will ignore. Returns the message or empty string.

`__init__(self, parent=None)`

Module: `utils/pymol_objects.py`

Type: `PymolObjects` method

objects(self)

Module: `utils/pymol_objects.py`

Type: `PymolObjects` method

The object list as of the last poll.

refresh(self)

Module: `utils/pymol_objects.py`

Type: `PymolObjects` method

Re-read PyMOL's object list and emit `changed` if it moved.

stop(self)

Module: `utils/pymol_objects.py`

Type: `PymolObjects` method

Stop polling. Called from `CTDialog.closeEvent` so a hidden dialog leaves no live timer.

get_relation_type_vector(mat, index, relation_type, numbering)

Module: `utils/relations.py`

Per-residue participation in one class of contact-contact relation.

bucket_bounds(relation_vector, n_buckets=BUCKETS)

Module: `utils/relations.py`

Upper bounds of the colour buckets, taken from the quantiles of the non-zero values. Zero is excluded because it gets its own white level and never shares a bucket.

register_shades()

Module: `utils/relations.py`

Define every shade colour, blended from white towards each relation-type hue (once per session).

_shade_levels(n_bounds)

Module: `utils/relations.py`

Pick `n_bounds` shades spread across the registered range, so the top is always full.

_shade_fraction(level, of)

Module: `utils/relations.py`

How far towards the hue a given level sits. Level 0 is white; the rest start at `MIN_SHADE`.

color_by_relation(molecule_name, relation_vector, numbering, relation_type, bounds=None)

Module: `utils/relations.py`

Colour a PyMOL object by a relation vector, in discrete levels.

make_scale_bar(topo_obj, relation_type, bounds)

Module: `utils/relations.py`

Put a stepped, labelled colour bar in the viewer so the numbers behind the colours show.

Stepped because the colouring is, so what the legend shows is what the structure got.

get_residue_range(self, obj_name)

Module: `utils/residues.py`

Retrieves the residue range for each chain in the specified object. Updates the chain combo box and residue range spinbox.

update_residue_range(self)

Module: `utils/residues.py`

Updates the residue range spinbox based on the currently selected chain.

_refresh_objects(self)

Module: `utils/trajectory.py`

Ask the shared object model to re-poll. Fallback to `update_list()`.

select_mol_file(self)

Module: `utils/trajectory.py`

Opens a file dialog to select a structure file (PDB or CIF) for trajectory analysis. Loads the file into PyMOL.

select_xtc_file(self)

Module: `utils/trajectory.py`

Opens a file dialog to select a trajectory file (XTC). Loads the trajectory into PyMOL and reports the non-polymer atoms the analysis will ignore.

export_frames_from_traj(self)

Module: `utils/trajectory.py`

Exports each frame of the loaded trajectory as a separate PDB file. Prompts the user for confirmation and an output directory.

update_output_widgets_multi(self)

Module: `utils/updates.py`

Updates the visibility of output widgets in the multi-file analysis tab based on checkbox states.

update_output_widgets_local(self)

Module: `utils/updates.py`

Updates the visibility of output widgets in the local analysis tab based on checkbox states.

update_output_widgets(self)

Module: `utils/updates.py`

Update visibility of output widgets for single-file export options.

update_local_list(self, new_objects=None)

Module: `utils/updates.py`

Updates the list of available objects in the local analysis dropdown.

update_list(self, new_objects=None)

Module: `utils/updates.py`

Updates the list of available objects in the single-file analysis dropdown.

is_placeholder_object(obj_name)

Module: `utils/validation.py`

Return True when a combo-box value is not a real PyMOL object.

legalize_object_name(raw_name)

Module: `utils/validation.py`

Return a PyMOL-safe object name for explicit load operations.

object_exists(obj_name)

Module: `utils/validation.py`

Return True when the named object exists in the current PyMOL session.

object_selection(obj_name)

Module: `utils/validation.py`

Return an exact object selection for a PyMOL object name.

polymer_selection(obj_name)

Module: `utils/validation.py`

Return the analysable part of an object: its polymer atoms only (replaces the old 'remove non-polymer' step).

chain_selection(obj_name, chain_id)

Module: `utils/validation.py`

Return a PyMOL selection for one polymer chain of an object.

selection_has_atoms(selection)

Module: `utils/validation.py`

Return True when a PyMOL selection currently contains atoms.

get_object_chains(obj_name)

Module: `utils/validation.py`

Return the polymer chains of an existing object, or an empty list on failure.

count_object_states(obj_name)

Module: `utils/validation.py`

Return the number of coordinate states for an object (0 if it doesn't exist).

is_trajectory(obj_name)

Module: `utils/validation.py`

Return True when an object has more than one state (a trajectory or NMR ensemble).

validate_structure_file(path_like)

Module: `utils/validation.py`

Validate a PDB/CIF file path and return it as a Path.

validate_trajectory_file(path_like)

Module: `utils/validation.py`

Validate a supported trajectory file path and return it as a Path.

natural_key(path)

Module: `utils/validation.py`

Sort key that reads runs of digits as numbers, so frame2 comes before frame10.

list_structure_files(directory)

Module: `utils/validation.py`

Return the PDB/CIF files of a directory, ordered by name with numbers read as numbers.

set_frame_spinbox_bounds(spinbox, file_count)

Module: `utils/validation.py`

Set a 1-based frame spinbox range without creating invalid Qt ranges.

selected_frame_file(files, frame_index)

Module: `utils/validation.py`

Return the file for a 1-based frame index or raise a clear error.

GUI Functions

_show_admin_required_dialog()

Module: `__init__.py`

Show a Qt message box explaining that elevated privileges are required.

_show_restart_required_dialog()

Module: `__init__.py`

Tell the user dependencies were installed but PyMOL must be restarted.

_try_register()

Module: `__init__.py`

Attempt to register plugin functions and add the GUI menu item.

__init_plugin__(app=None)

Module: `__init__.py`

Initialize the plugin within PyMOL.

Invoked by PyMOL when the plugin is loaded. The flow is:

1. Register PyMOL commands if dependencies are already present.

2. Otherwise, if PyMOL is installed system-wide and this session lacks administrator/root privileges, show guidance and stop (a writable environment is required to install).
3. Install the missing dependencies into PyMOL's own conda environment.
4. Invalidate the import cache and re-register. If the freshly installed packages are not importable in this live session, ask the user to restart PyMOL.

run_plugin_gui()

Module: `__init__.py`

Create or raise the plugin GUI dialog.

This function is bound as the menu action entrypoint and returns the singleton dialog instance so callers (and tests) can interact with it.

__init__(self, parent=None)

Module: `gui_class.py`

Type: `CTDialog` method

Construct the dialog: window chrome, the three tabs, then the shared object model.

_on_objects_changed(self, objects)

Module: `gui_class.py`

Type: `CTDialog` method

Refresh both object dropdowns from one polled list.

closeEvent(self, event)

Module: `gui_class.py`

Type: `CTDialog` method

Stop polling when the dialog closes.

init_ui(self)

Module: `gui_class.py`

Type: `CTDialog` method

Create the top-level tab widget and initialize each feature tab.

__init__(self, dialog, parent=None)

Module: `tabs/local_tab.py`

Type: `LocalTab` method

pymol_objects(self)

Module: `tabs/local_tab.py`

Type: `LocalTab` method

get_local_values(self)

Module: `tabs/local_tab.py`

Type: `LocalTab` method

Every setting this tab exposes.

clear_selected_local_file(self)

Module: tabs/local_tab.py

Type: LocalTab method

choose_local_file(self)

Module: tabs/local_tab.py

Type: LocalTab method

choose_local_output_dir(self)

Module: tabs/local_tab.py

Type: LocalTab method

handle_local_object_change(self, obj_name=None)

Module: tabs/local_tab.py

Type: LocalTab method

Log what analysis will ignore, then refresh the chain/residue controls.

get_residue_range(self, obj_name=None)

Module: tabs/local_tab.py

Type: LocalTab method

update_residue_range(self)

Module: tabs/local_tab.py

Type: LocalTab method

update_chain_combo_box(self)

Module: tabs/local_tab.py

Type: LocalTab method

update_output_widgets_local(self)

Module: tabs/local_tab.py

Type: LocalTab method

run_local_ct(self)

Module: tabs/local_tab.py

Type: LocalTab method

update_local_list(self)

Module: tabs/local_tab.py

Type: LocalTab method

_build_local_input_group(self)

Module: tabs/local_tab.py

`_build_local_params_group(self)`

Module: tabs/local_tab.py

`_build_local_analysis_group(self)`

Module: tabs/local_tab.py

`_build_local_export_group(self)`

Module: tabs/local_tab.py

`init_local_tab(self)`

Module: tabs/local_tab.py

Initializes the 'Local Circuit Topology' tab in the GUI. Sets up widgets for file selection, parameter input, analysis options, and exporting.

`__init__(self, dialog, parent=None)`

Module: tabs/multiple_file_tab.py

Type: MultiFileTab method

`pymol_objects(self)`

Module: tabs/multiple_file_tab.py

Type: MultiFileTab method

The shared object model, so utils.trajectory can request a refresh.

`get_multiple_values(self)`

Module: tabs/multiple_file_tab.py

Type: MultiFileTab method

Every setting this tab exposes.

`choose_input_dir_multi(self)`

Module: tabs/multiple_file_tab.py

Type: MultiFileTab method

`choose_output_dir_multi(self)`

Module: tabs/multiple_file_tab.py

Type: MultiFileTab method

`select_mol_file(self)`

Module: tabs/multiple_file_tab.py

Type: MultiFileTab method

`select_xtc_file(self)`

Module: tabs/multiple_file_tab.py

Type: MultiFileTab method

`export_frames_from_traj(self)`

Module: tabs/multiple_file_tab.py

Type: MultiFileTab method

`update_output_widgets_multi(self)`

Module: tabs/multiple_file_tab.py

Type: MultiFileTab method

`run_multi_analysis(self)`

Module: tabs/multiple_file_tab.py

Type: MultiFileTab method

`run_single_frame_analysis(self)`

Module: tabs/multiple_file_tab.py

Type: MultiFileTab method

`toggle_frame_controls(self, enabled)`

Module: tabs/multiple_file_tab.py

Type: MultiFileTab method

`update_list(self)`

Module: tabs/multiple_file_tab.py

Type: MultiFileTab method

Legacy name still called by helper code; routes through the shared model.

`_build_multi_dir_group(self)`

Module: tabs/multiple_file_tab.py

`_build_multi_traj_group(self)`

Module: tabs/multiple_file_tab.py

`_build_multi_params_group(self)`

Module: tabs/multiple_file_tab.py

`_build_multi_frame_widgets(self, layout)`

Module: tabs/multiple_file_tab.py

`_build_multi_filters_group(self)`

Module: tabs/multiple_file_tab.py

`_build_multi_plot_group(self)`

Module: tabs/multiple_file_tab.py

`_build_multi_export_group(self)`

Module: tabs/multiple_file_tab.py

init_multi_file_tab(self)

Module: tabs/multiple_file_tab.py

Initializes the 'Multi-File Analysis' tab in the GUI. Sets up widgets for directory selection, trajectory handling, parameter input, analysis options, and exporting.

__init__(self, dialog, parent=None)

Module: tabs/single_file_tab.py

Type: SingleFileTab method

pymol_objects(self)

Module: tabs/single_file_tab.py

Type: SingleFileTab method

get_values(self)

Module: tabs/single_file_tab.py

Type: SingleFileTab method

Every setting this tab exposes.

get_vis_vals(self)

Module: tabs/single_file_tab.py

Type: SingleFileTab method

clear_selected_single_file(self)

Module: tabs/single_file_tab.py

Type: SingleFileTab method

choose_file(self)

Module: tabs/single_file_tab.py

Type: SingleFileTab method

choose_output_dir(self)

Module: tabs/single_file_tab.py

Type: SingleFileTab method

handle_standard_object_change(self, obj_name=None)

Module: tabs/single_file_tab.py

Type: SingleFileTab method

Log what analysis will ignore. Non-polymer atoms need no action from the user.

update_output_widgets(self)

Module: tabs/single_file_tab.py

Type: SingleFileTab method

visualize_molecule(self, contact_type)

Module: tabs/single_file_tab.py

Type: SingleFileTab method

run_standard_analysis(self)

Module: tabs/single_file_tab.py

Type: SingleFileTab method

update_list(self)

Module: tabs/single_file_tab.py

Type: SingleFileTab method

_build_input_group(self)

Module: tabs/single_file_tab.py

_build_params_group(self)

Module: tabs/single_file_tab.py

_build_plot_group(self)

Module: tabs/single_file_tab.py

_build_vis_group(self)

Module: tabs/single_file_tab.py

_build_export_group(self)

Module: tabs/single_file_tab.py

_build_folding_group(self)

Module: tabs/single_file_tab.py

init_single_file_tab(self)

Module: tabs/single_file_tab.py

Initializes the 'Single-File Analysis' tab in the GUI. Sets up widgets for file selection, parameter input, analysis options, visualization, and exporting.

Initialization Functions

is_path_user(path)

Module: initialization_checks.py

Checks if a given path is within the user's home directory.

is_running_as_admin()

Module: initialization_checks.py

Returns True if the current process has administrator / root privileges.

On Windows this calls `IsUserAnAdmin()`; on POSIX it checks `effective uid == 0`. Returns `False` on any error so callers can treat the result conservatively.

`install_failed(reqs=REQUIREMENTS_FILE)`

Module: `initialization_checks.py`

Prints instructions for manual installation of dependencies if automated installation fails.

`_normalize_requirement_name(requirement)`

Module: `initialization_checks.py`

Convert a conda-style dependency entry into its package name.

`requirement_specs(req_path)`

Module: `initialization_checks.py`

Parse `requirements.yml` into package name -> the exact spec the file declares.

`check_installed_packages(requirements_list)`

Module: `initialization_checks.py`

Checks if the required packages are installed in the current environment.

`_is_package_available(pack)`

Module: `initialization_checks.py`

Check whether a conda-named package is importable in this interpreter.

`is_conda_installed()`

Module: `initialization_checks.py`

Checks if conda is discoverable on the system PATH.

`_find_conda_executable()`

Module: `initialization_checks.py`

Locate the conda executable that owns the running PyMOL environment.

`_installed_spec(package)`

Module: `initialization_checks.py`

Pin `package` to the version already installed in this prefix, read from `conda-meta`.

`_run_conda(command)`

Module: `initialization_checks.py`

Run a conda command, returning `None` if it could not be launched at all.

`_retry_without_broken_pins(command)`

Module: `initialization_checks.py`

Retry a conda command with a malformed `conda-meta/pinned` moved out of the way.

`install_dependencies(reqs=REQUIREMENTS_FILE, missing=None)`

Module: `initialization_checks.py`

Install plugin dependencies into PyMOL's own conda environment.

register_pymol_functions()

Module: initialization_checks.py

Register the plugin's core functions as PyMOL commands.

This function imports the plugin's modular command wrappers and extends the PyMOL `cmd` object so that the commands become available to users as top-level PyMOL commands.

Last Updated: September 20, 2026 (plugin v0.0.4)